

Digital Signal Processing Lab

Experiment 1 : Sampling

Group 06 (WEDNESDAY)



N DEEKSHITHA

24EC10105

SHRAVI BANSAL

24EC10130

Objectives

In this experiment, we need to perform:

- Sampling of a Sinusoidal Waveform
- Sampling at below Nyquist rate and effect of aliasing
- Spectrum of a Square Wave
- Interpolation or Upsampling

PART A: Sampling of a Sinusoidal Waveform

CODE:

```
close all;

clc;

%(a)sampling of sinusodial waveform

% Defining continuous time vector and continuous signal vector

t_cont = 0:1e-5:0.01;

x_cont = 10*cos(2*pi*1e3*t_cont) + 6*cos(2*pi*2e3*t_cont) + 2*cos(2*pi*4e3*t_cont);

% Sampling frequency and sampling period

Fs = 12e3;

Ts = 1/Fs;

% Creating sample points in time vector indices closest to desired sampling instants

sample_times = 0:Ts:(255*Ts);

% Finding the indices in the continuous time vector closest to the sample times

[~, sample_indices] = arrayfun(@(x) min(abs(t_cont - x)), sample_times);

% using the syntax given in matlab documentation

% Sampling the continuous signal vector using those indices

x_sampled_from_vector = x_cont(sample_indices);

% Plotting

subplot(4,2,1)

stem(sample_times, x_sampled_from_vector, 'r');

hold on;

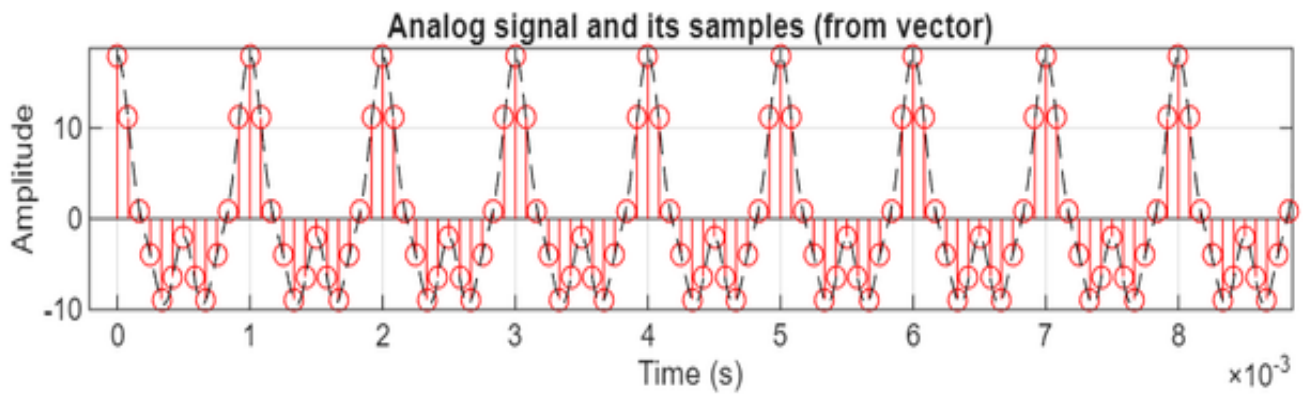
plot(t_cont, x_cont, 'k--');

title('Analog signal and its samples (from vector)');

xlabel('Time (s)');

ylabel('Amplitude');

grid on;
```



```
%% DFT
```

```
N = 12;
```

```
figure;
```

```
X_n = x_sampled_from_vector(1:N);
```

```
X = fft(X_n);
```

```
f = (-N/2:N/2-1)*Fs/N;
```

```
X_shifted = fftshift(X);
```

```
subplot(4,2,1);
```

```
stem(f, abs(X_shifted)/N, 'filled');
```

```
title('Fs = 12KHz , N = 12');
```

```
xlabel('Frequency');
```

```
ylabel('|X(f)|/N');
```

```
N = 64;
```

```
X_n = x_sampled_from_vector(1:N);
```

```
X = fft(X_n);
```

```
f = (-N/2:N/2-1)*Fs/N;
```

```
X_shifted = fftshift(X);
```

```
subplot(4,2,2);
```

```
stem(f, abs(X_shifted)/N, 'filled');
```

```
title('Fs = 12KHz , N = 64');
```

```
xlabel('Frequency');
```

```
ylabel('|X(f)|/N');
```

```
N = 128;
```

```
X_n = x_sampled_from_vector(1:N);
```

```

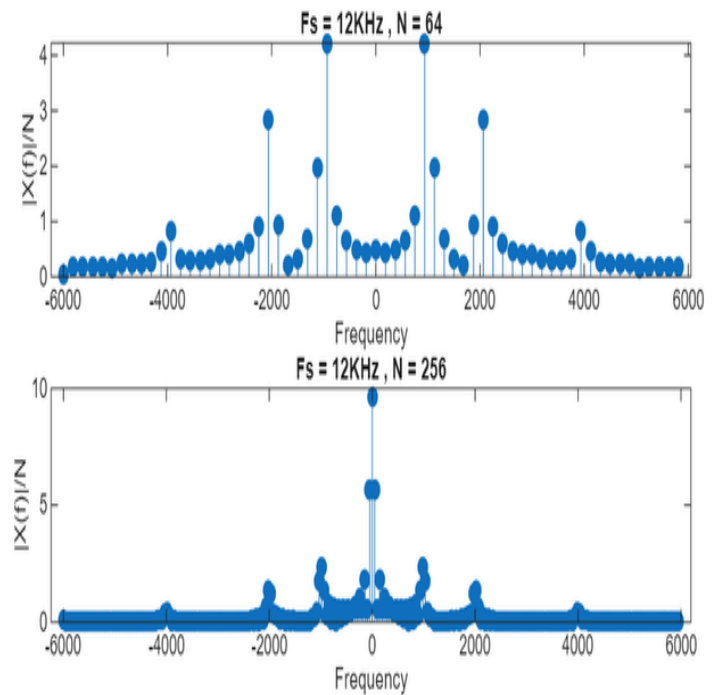
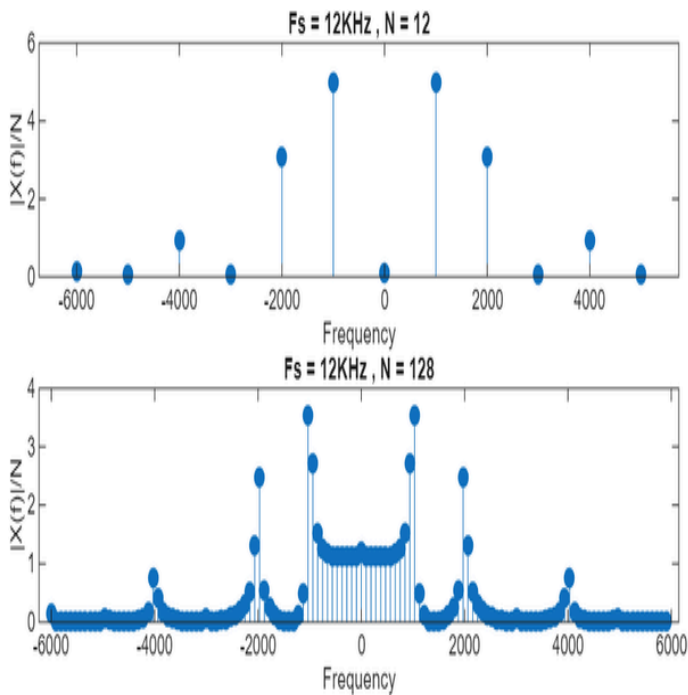
X = fft(X_n);
f = (-N/2:N/2-1)*Fs/N;
X_shifted = fftshift(X);
subplot(4,2,3);
stem(f, abs(X_shifted)/N, 'filled');
title('Fs = 12KHz , N = 128');
xlabel('Frequency');
ylabel('|X(f)|/N');

```

```

N = 256;
X_n = x_sampled_from_vector(1:N);
X = fft(X_n);
f = (-N/2:N/2-1)*Fs/N;
X_shifted = fftshift(X);
subplot(4,2,4);
stem(f, abs(X_shifted)/N, 'filled');
title('Fs = 12KHz , N = 256');
xlabel('Frequency');
ylabel('|X(f)|/N');

```



PART B: Sampling at below Nyquist rate and effect of aliasing

CODE:

```
%part a done

%%(b) Sampling at below nyquist rate and effect of aliasing

% continuous-time signal

t_cont = 0:1e-5:0.01;

x_cont = 10*cos(2*pi*1e3*t_cont) + 6*cos(2*pi*2e3*t_cont) + 2*cos(2*pi*4e3*t_cont);

% 2. Sampling parameters below Nyquist

Fs = 8e3; % 8 kHz < Nyquist (>8 kHz)

Ts = 1/Fs;

n = 0:255;

% sampling times

sample_times = n * Ts;

% closest indices in continuous time vector

[~, sample_indices] = arrayfun(@(x) min(abs(t_cont - x)), sample_times);

% Sample continuous signal vector at those indices (vector form sampling)

x_sampled_from_vector = x_cont(sample_indices);

% Plot original continuous signal and sampled points

figure;

subplot(3,1,1);

plot(t_cont, x_cont, 'k', 'LineWidth', 1.5); hold on;

stem(sample_times, x_sampled_from_vector, 'r', 'filled');

title('Sampling from Vector Below Nyquist Rate (8 kHz)');

xlabel('Time (s)');

ylabel('Amplitude');

legend('Original Signal', 'Sampled Points');

grid on;

% Compute FFT of the sampled signal

N = length(x_sampled_from_vector);

X = fft(x_sampled_from_vector, N);

f = (0:N-1)*(Fs/N); % Frequency vector

% Plot magnitude spectrum showing aliasing

subplot(3,1,2);
```

```

plot(f, abs(X)/N, 'b', 'LineWidth', 1.5);

title('Magnitude Spectrum of Sampled Signal (Aliasing)');

xlabel('Frequency (Hz)');

ylabel('|X(f)|');

xlim([0 Fs/2]); % Up to Nyquist frequency

grid on;

% Reconstruct the signal in time domain using inverse FFT
x_reconstructed = ifft(X, N, 'symmetric');

% Plot reconstructed signal and compare to original
subplot(3,1,3);

plot(n*Ts, x_reconstructed, 'r-', 'LineWidth', 1.5); hold on;

plot(t_cont, x_cont, 'k--');

title('Reconstructed Signal from Samples (Aliasing Distortion)');

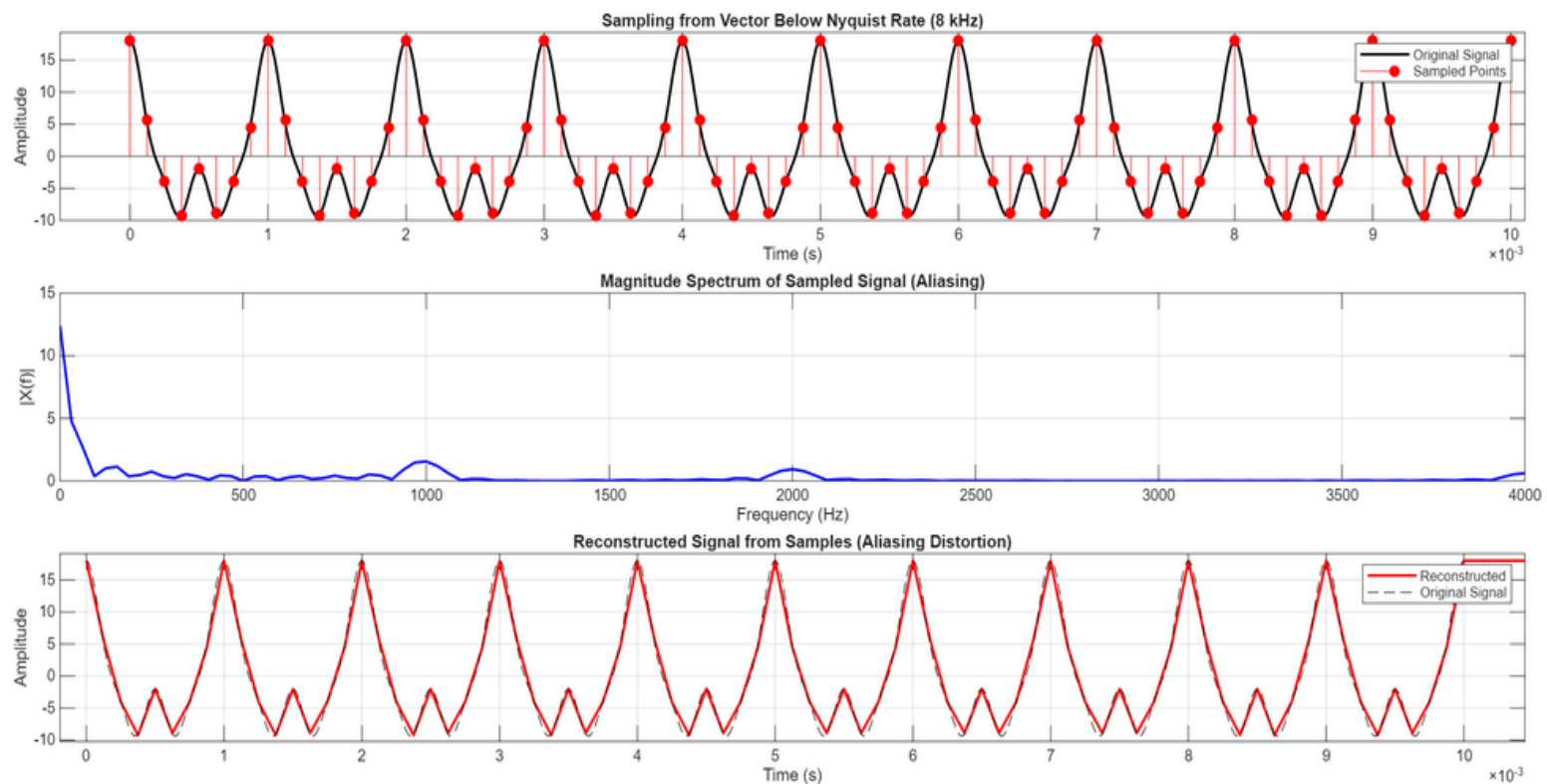
xlabel('Time (s)');

ylabel('Amplitude');

legend('Reconstructed', 'Original Signal');

grid on;

```



```

% Sampling parameters below Nyquist
Fs = 5e3; % 5 kHz < Nyquist (8 kHz)

Ts = 1/Fs;

```

```

n = 0:255;

% Define sampling times
sample_times = n * Ts;

% closest indices in continuous time vector
[~, sample_indices] = arrayfun(@(x) min(abs(t_cont - x)), sample_times);

% Sample continuous signal vector at those indices (vector form sampling)
x_sampled_from_vector = x_cont(sample_indices);

% Plot original continuous signal and sampled points
figure;

subplot(3,1,1);

plot(t_cont, x_cont, 'k', 'LineWidth', 1.5); hold on;
stem(sample_times, x_sampled_from_vector, 'r', 'filled');
title('Sampling from Vector Below Nyquist Rate (5 kHz)');
xlabel('Time (s)');
ylabel('Amplitude');
legend('Original Signal', 'Sampled Points');
grid on;

% Compute FFT of the sampled signal
N = length(x_sampled_from_vector);
X = fft(x_sampled_from_vector, N);
f = (0:N-1)*(Fs/N); % Frequency vector

% Plot magnitude spectrum showing aliasing
subplot(3,1,2);

plot(f, abs(X)/N, 'b', 'LineWidth', 1.5);
title('Magnitude Spectrum of Sampled Signal (Aliasing)');
xlabel('Frequency (Hz)');
ylabel('|X(f)|');
xlim([0 Fs/2]); % Up to Nyquist frequency
grid on;

% Reconstruct the signal in time domain using inverse FFT
x_reconstructed = ifft(X, N, 'symmetric');

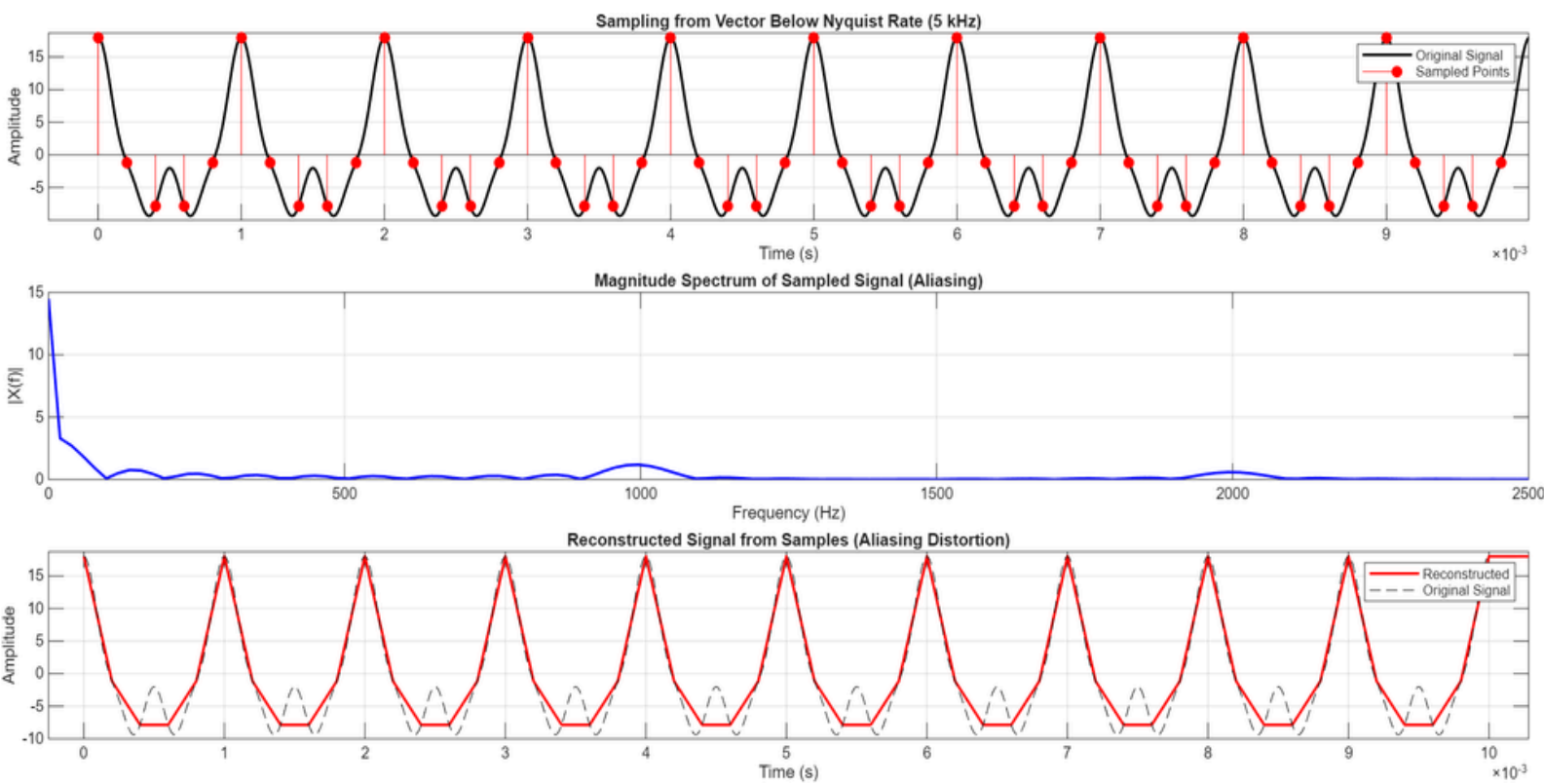
% Plot reconstructed signal and compare to original
subplot(3,1,3);

```

```

plot(n*Ts, x_reconstructed, 'r-', 'LineWidth', 1.5); hold on;
plot(t_cont, x_cont, 'k--');
title('Reconstructed Signal from Samples (Aliasing Distortion)');
xlabel('Time (s)');
ylabel('Amplitude');
legend('Reconstructed', 'Original Signal');
grid on;

```



```

% Sampling parameters below Nyquist
Fs = 4e3; % 4 kHz < Nyquist (8 kHz)
Ts = 1/Fs;
n = 0:255;

% Define sampling times
sample_times = n * Ts;

% closest indices in continuous time vector
[~, sample_indices] = arrayfun(@(x) min(abs(t_cont - x)), sample_times);

% Sample continuous signal vector at those indices (vector form sampling)
x_sampled_from_vector = x_cont(sample_indices);

% Plot original continuous signal and sampled points

figure;
subplot(3,1,1);

```



```

plot(t_cont, x_cont, 'k', 'LineWidth', 1.5); hold on;
stem(sample_times, x_sampled_from_vector, 'r', 'filled');
title('Sampling from Vector Below Nyquist Rate (4 kHz)');
xlabel('Time (s)');
ylabel('Amplitude');
legend('Original Signal', 'Sampled Points');
grid on;

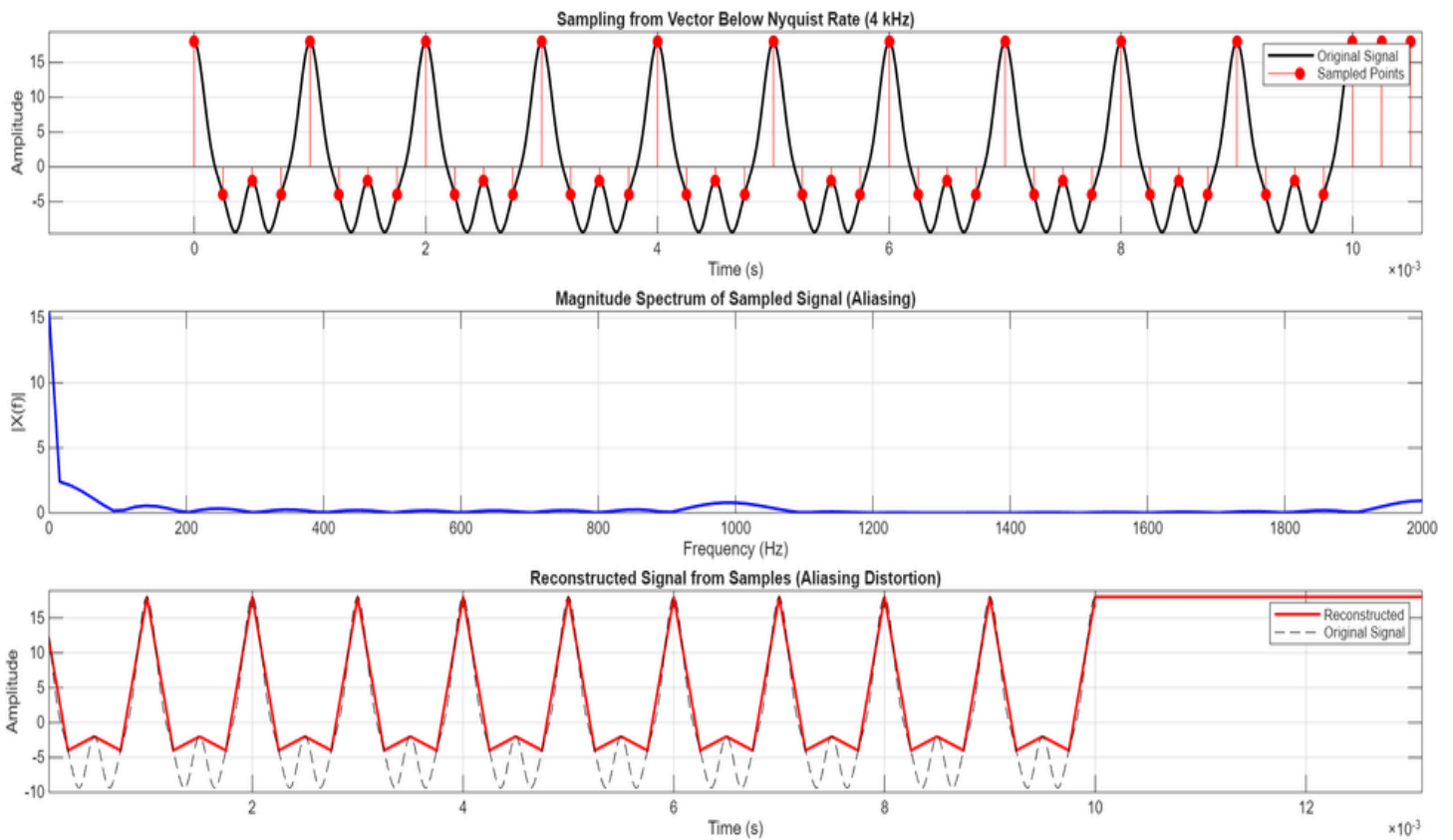
% Compute FFT of the sampled signal
N = length(x_sampled_from_vector);
X = fft(x_sampled_from_vector, N);
f = (0:N-1)*(Fs/N); % Frequency vector

% Plot magnitude spectrum showing aliasing
subplot(3,1,2);
plot(f, abs(X)/N, 'b', 'LineWidth', 1.5);
title('Magnitude Spectrum of Sampled Signal (Aliasing)');
xlabel('Frequency (Hz)');
ylabel('|X(f)|');
xlim([0 Fs/2]); % Up to Nyquist frequency
grid on;

% Reconstruct the signal in time domain using inverse FFT
x_reconstructed = ifft(X, N, 'symmetric');

% Plot reconstructed signal and compare to original
subplot(3,1,3);
plot(n*Ts, x_reconstructed, 'r-', 'LineWidth', 1.5); hold on;
plot(t_cont, x_cont, 'k--');
title('Reconstructed Signal from Samples (Aliasing Distortion)');
xlabel('Time (s)');
ylabel('Amplitude');
legend('Reconstructed', 'Original Signal');
grid on;

```



PART C: Spectrum of a Square Wave

CODE:

```

%%(c)Spectrum of square wave

% Parameters

fs = 20e3;           % Sampling frequency (Hz)

T = 1e-3;           % Square wave period (1 ms = 1 kHz)

t_10 = 10 * T;      % Time duration for 10 periods

f0 = 1e3;           % Fundamental frequency (Hz)

% Prepare time vectors

t = 0:1/fs:t_10 - 1/fs; % continuous time for 10 cycles

% Manually generate the square wave (no toolbox needed)

sq_wave = sign(sin(2*pi*f0*t));

% Sampled version (N = 256)

N = 256;

n = 0:N-1;

ts = 1/fs;

sq_sampled = sign(sin(2*pi*f0*n*ts));

% Magnitude spectrum using FFT

```

```

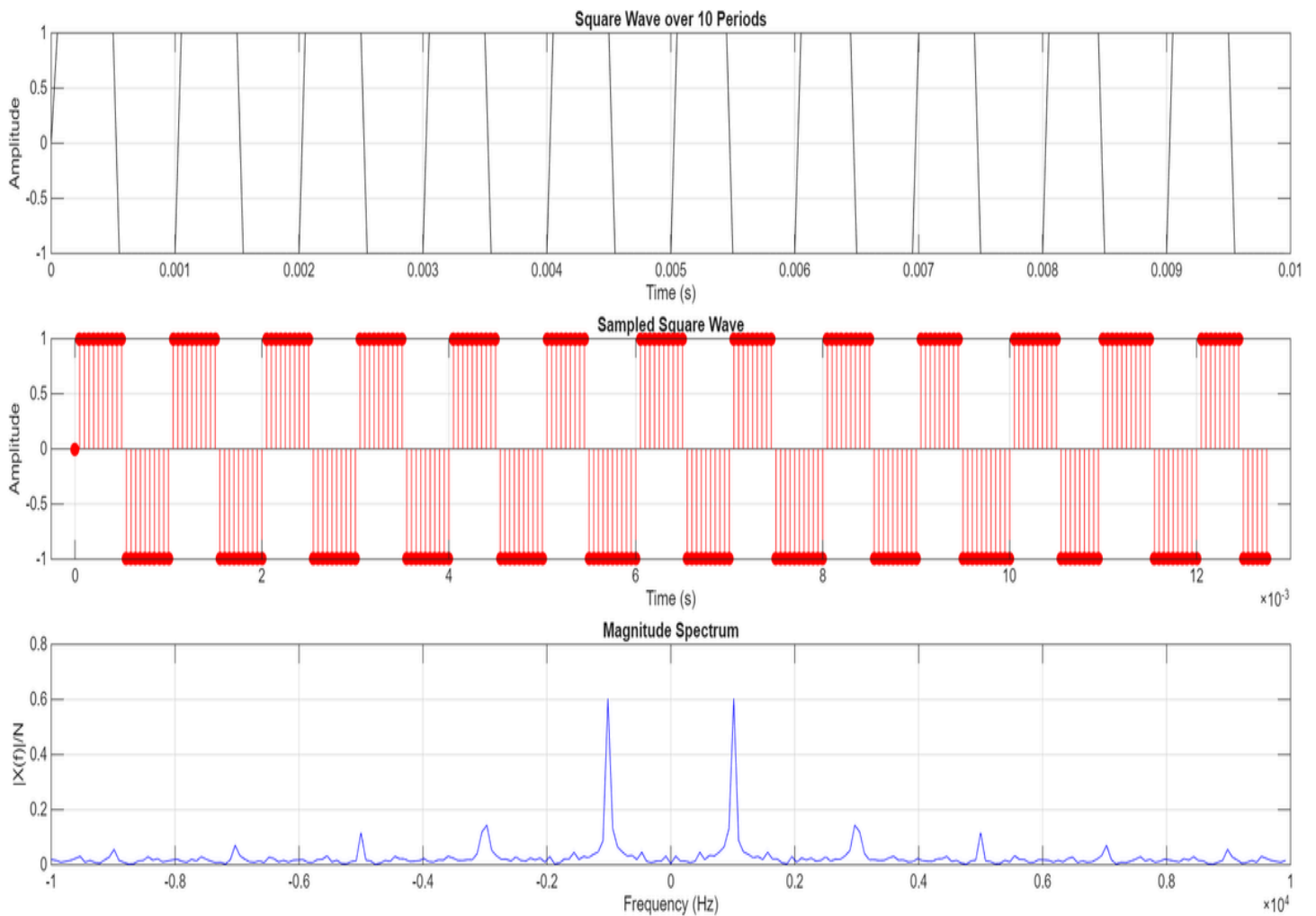
sq_n = sq_sampled(1:N);
sq_fft = fft(sq_n);
sq_shifted = fftshift(sq_fft);
f = (-N/2:N/2-1) * fs / N;
figure;

% 1. Continuous Square Wave
subplot(3,1,1)
plot(t, sq_wave, 'k')
title('Square Wave over 10 Periods')
xlabel('Time (s)')
ylabel('Amplitude')
grid on

% 2. Sampled Square Wave (N = 256)
subplot(3,1,2)
stem(n*ts, sq_sampled, 'r', 'filled')
title('Sampled Square Wave')
xlabel('Time (s)')
ylabel('Amplitude')
grid on

% 3. Magnitude Spectrum
subplot(3,1,3)
plot(f, abs(sq_shifted)/N, 'b')
title('Magnitude Spectrum')
xlabel('Frequency (Hz)')
ylabel('|X(f)|/N')
ylim([0 0.8]);
grid on

```



PART D: Interpolation or Upsampling

CODE:

```

%(d)interpolation or upsampling

% Parameters

fs = 12000; % Original sampling rate

fs2 = fs*2; % Upsampled rate

fc = 6000; % Cutoff frequency

t1 = 0:1/fs:2e-3; % time at Fs1

t2 = 0:1/fs2:2e-3; % time at Fs2 (for continuous plot)

% Original continuous-like signal

x_cont = 10*cos(2*pi*1e3*t2) + 6*cos(2*pi*2e3*t2) + 2*cos(2*pi*4e3*t2);

% Sampled at 12 kHz

x1 = 10*cos(2*pi*1e3*t1) + 6*cos(2*pi*2e3*t1) + 2*cos(2*pi*4e3*t1);

% Step 3: Insert one zero after each sample

x_up = zeros(1, length(x1)*2);
  
```

```

x_up(1:2:end) = x1;

% Time vector for upsampled signal
t_new = 0 : 1/fs2 : (length(x_up)-1)/fs2;

% Lowpass filter
[b,a] = butter(6, fc/(fs2/2)); % 6th order LPF
x_filt = filter(b,a,x_up);

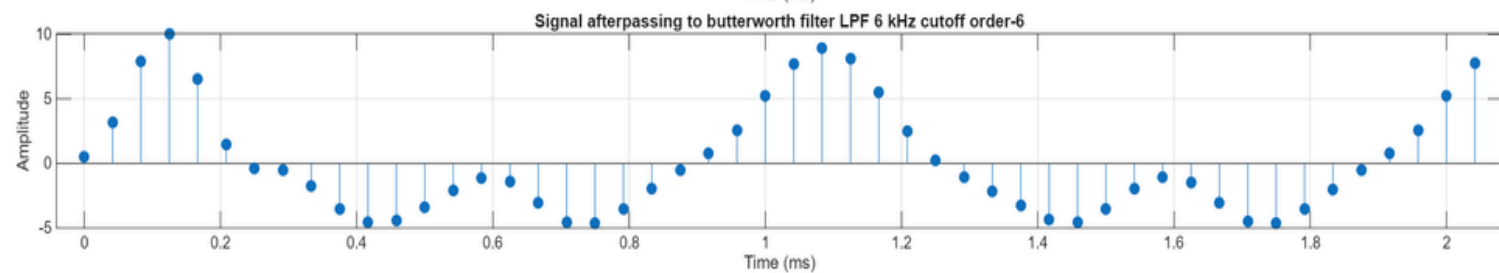
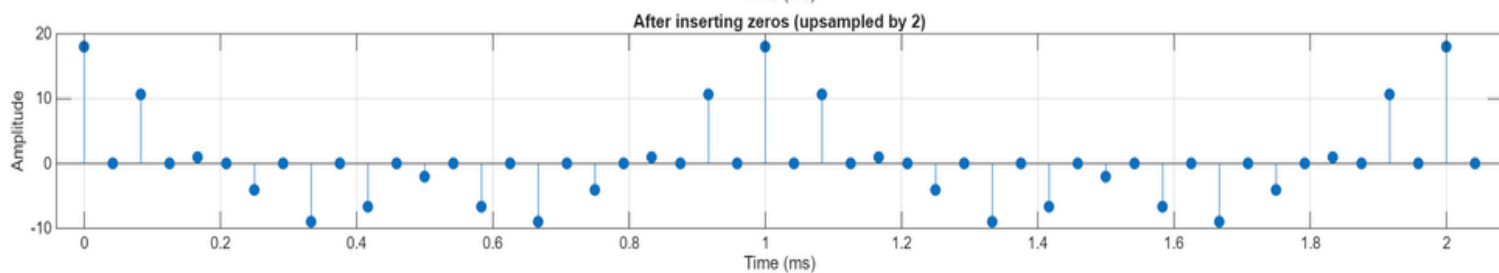
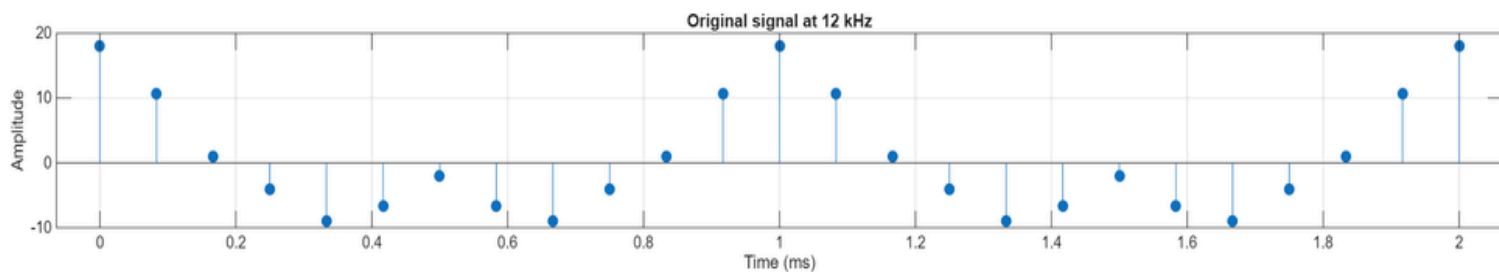
% Plot
figure;

subplot(3,1,1);
stem(t1*1e3, x1, 'filled');
title('Original signal at 12 kHz');
xlabel('Time (ms)'); ylabel('Amplitude'); grid on;

subplot(3,1,2);
stem(t_new*1e3, x_up, 'filled');
title('After inserting zeros (upsampled by 2)');
xlabel('Time (ms)'); ylabel('Amplitude'); grid on;

subplot(3,1,3);
stem(t_new*1e3, x_filt, 'filled');
title('Signal after passing to butterworth filter LPF 6 kHz cutoff order-6');
xlabel('Time (ms)'); ylabel('Amplitude'); grid on;

```



RESULTS:

SHRAVI BANSAL (24EC10130) :

PART A: Sampling of a Sinusoidal Waveform

When N is increased the changes in the spectrum are:

1. Improved Frequency Resolution
 - The frequency bin spacing decreases according to $\Delta f = F_s/N$.

This allows closely spaced frequency components to be distinguished more clearly.

1. Sharper Spectral Peaks
 - The frequency components in the spectrum appear narrower and more well-defined.
 - The true sinusoidal frequencies align more accurately with FFT bin positions.
2. Smoother Spectrum Appearance
 - The magnitude spectrum becomes less jagged and more continuous-looking.
 - Small changes in amplitude between adjacent frequency bins become more visible.
 - More DFT points mean the plotted spectrum will be less "blocky" and the peaks will be more accurately positioned and shaped. With a small N, spectral peaks may look wider, less defined, or not centered at expected frequency values.
3. Better Peak Identification
 - Higher N helps to more precisely locate the exact frequency of peaks.
 - The amplitudes of the peaks also appear closer to their theoretical values due to finer bin alignment.

PART B: Sampling at below Nyquist rate and effect of aliasing

Aliases of original frequencies present in $x(t)$ when the sampling rate is below Nyquist rate:

1. $F_s = 8 \text{ kHz}$

Nyquist frequency $F_s/2 = 4 \text{ kHz}$.

Frequencies above 4 kHz will fold back (alias) into the lower 0–4 kHz range.

With ideal filtering, all signal components are preserved in your sampled spectrum: will see peaks at 1 kHz, 2 kHz, and 4 kHz. There are no aliases of these original frequencies folding into lower-frequency bins.

1. $F_s = 5 \text{ kHz}$

Nyquist frequency $F_s/2 = 2.5 \text{ kHz}$.

Frequencies above 2.5 kHz will fold back (alias) into the lower 0–2.5 kHz range.

The 4 kHz component *does not* appear at 4 kHz! Instead, it folds back to 1 kHz, adding to the 1 kHz peak. Spectrum shows a stronger 1 kHz peak because both the original 1 kHz and aliased 4 kHz fold into the same bin.

1. $F_s = 4 \text{ kHz}$

Nyquist frequency $F_s/2 = 2 \text{ kHz}$.

Frequencies above 2 kHz will fold back (alias) into the lower 0–2 kHz range.

The 4 kHz component *does not* appear at 4 kHz! Instead, it folds back to 1 kHz, adding to the 1 kHz peak. Spectrum shows a stronger 1 kHz peak because both the original 1 kHz and aliased 4 kHz fold into the same bin.

The 1 kHz and 2 kHz tones will appear normally in the sampled signal spectrum.

The 4 kHz tone folds back to DC (0 Hz), which manifests as a constant offset in the sampled signal.

This aliasing distorts the signal spectrum, mixing high-frequency signals into low-frequency components (here, DC).

Signal reconstruction from samples taken at 4 kHz is distorted because the 4 kHz component is incorrectly represented at 0 Hz.

Hence, by reconstructed signal only the signal sampled at 8kHz matches the original best out of the three.

PART C: Spectrum of a Square Wave

- Peaks at odd harmonics: We can see sharp peaks at the fundamental (1 kHz), third harmonic (3 kHz), fifth harmonic (5 kHz), etc. These are the nonzero components of the square wave's spectrum.

PART D: Interpolation or Upsampling

- The sampled signal accurately represents the analog signal without aliasing, since the sampling frequency is greater than twice the highest frequency component (Nyquist criterion).

Delay and Scaling:

The filtered output is slightly delayed compared to the original due to the filter's group delay. Minor amplitude reduction occur because of the filter's passband attenuation (usually small for Butterworth with moderate order).

N DEEKSHITHA (24EC10105) :

PART A: Sampling of a Sinusoidal Waveform

- We have sampled the analog signal at the sampling frequency of 12KHz. Since the maximum frequency present in the signal is 4KHz and the sampling frequency is 12KHz which in turn satisfies the Nyquist criterion (where sampling frequency should be minimum 8KHz), we will be able to reconstruct the signal perfectly.
- Increasing the number of points for sampling the frequency domain (using N point DFT) reduces the spread around the required cosines and gives more accurate plots.
- As N tends to be infinite we could observe that the DFT of the signal approaches the Fourier Transform since the length of the frequency interval approaches zero and the frequency axis becomes continuous.

PART B: Sampling at below Nyquist rate and effect of aliasing

- It is necessary to sample the given signal strictly greater than twice the maximum frequency present in the given signal to ensure perfect recovery of the bandlimited signal.
- The signal cannot be reconstructed perfectly when sampled exactly at the Nyquist Rate in this case.

PART C: Spectrum of a Square Wave

- We know that square wave is a half wave symmetric signal, hence only odd harmonics will be present. This has been verified from the square wave spectra. If we observe more closely, the square wave in the time domain is not completely steep as it would require almost infinite frequencies to represent the sharp transition.

PART D: Interpolation or Upsampling

- Upsampling by N produces a higher-rate sequence by creating N samples in the original signal. Upsampling is simply a contraction of the frequency axis by a factor of N.

- In the time domain, the upsampled signal doesn't look "natural" since there are $N-1$ zeros between every sample drawn from the input. Thus a "smooth" input signal no longer looks smooth after upsampling. A solution would be to try to interpolate the original samples in order to "fill in" the gaps.

DISCUSSION

SHRAVI BANSAL (24EC10130) :

Our code samples the continuous-time signal by directly picking values from the precomputed vector `x_cont` at the points nearest to the sampling instants instead of computing the sampled signal from the cosine formula. This approach **preserves the sampled data integrity from the actual continuous vector signal representation rather than re-evaluating the formula at discrete points**.

Increasing N **increases the computational load** and memory usage of the FFT algorithm but reveals **more spectral detail**, a higher-resolution frequency spectrum, letting us distinguish frequency components more precisely, and produces smoother and sharper spectrum plots.

I understood that when we do an FFT of these samples: we only get frequencies between 0 and $F_s/2$ and any part of the original spectrum above $F_s/2$ is mirrored (folded) back inward around multiples of $F_s/2$, this "folded back" part is indistinguishable from true low-frequency components — that's aliasing.

Frequency (scaled and multiplied by 10^4), so it goes from -10 kHz to +10 kHz. This is because **the DFT of a real-valued signal is symmetric about zero**. For a square wave of frequency f_0 , only **odd harmonics** ($f_0, 3f_0, 5f_0, \dots$) appear, and their amplitudes decrease with harmonic order:

Amplitude of n th harmonic $\propto 1/n$

(where n is odd: 1, 3, 5...).

We insert zeros to increase the sample rate artificially, then apply a lowpass filter to fill in those zeros smoothly, thereby producing an upsampled, properly interpolated signal. The filtered output may be slightly delayed, which can increase if we increase the order, but increasing order is a **trade-off between delay, computation cost and more exact and beautiful reconstructed signal**. Can also have minor amplitude attenuation near the filter's cutoff frequency.

N DEEKSHITHA (24EC10105) :

We sampled the continuous-time signal directly by selecting values from the signal given, so the sampled points are close to the desired sampling points.

We have observed that it is necessary to sample the given signal strictly greater than twice the maximum frequency present in the given signal to ensure perfect recovery of the bandlimited signal

We also observed, by Increasing the FFT length N increases but improves spectral resolution, resulting finer frequency details, smoother and sharper spectrum plots, and allowing better distinction between close frequency components.

This is also observed that the FFT output contains frequencies only between 0 and $F_s/2$; components above $F_s/2$ fold back due to aliasing, becoming of lower frequencies. The DFT of a real-valued signal is symmetric about zero.

For a square wave of frequency f_0 , only odd harmonics appear, with amplitudes proportional to $1/n$ (odd n).

The upsampling was done by inserting zeroes between samples and applying a low-pass filter for interpolation. A higher original sampling rate would produce smoother, more continuous-looking waveforms, and improving visual clarity. And inserting zeroes between the samples is not required cause the result can be the same even without inserting zeroes. After inserting zeroes it is quite visible that f_s is changed.